

ПРАКТИЧЕСКАЯ РАБОТА № 1

МНОГОПОТОЧНОСТЬ И ПАРАЛЛЕЛЬНЫЕ ВЫЧИСЛЕНИЯ В JAVA

Platform Threads, Executors, CompletableFuture, ForkJoin и Virtual Threads

Дисциплина	Проектирование и разработка клиент-серверных приложений
Учебный год	2026/2027
Технологии	Java 25 LTS · Maven · JUnit 5 · Java Concurrency API
Результат	Четыре связанных задания и сравнительный эксперимент

Как работать с гайдом. Сначала повторите демонстрационные примеры и убедитесь, что они запускаются в вашем проекте. Затем выполните четыре задания, используя примеры как справочник по API, но самостоятельно проектируя классы и взаимодействие компонентов.

Ключевая идея. Механизм выполнения выбирают по характеру нагрузки. CPU-bound задачи ограничены вычислительными ресурсами, а blocking-задачи значительную часть времени ожидают внешний ресурс. Большое число потоков само по себе не гарантирует ускорения.

Содержание

Содержание	2
1. Цель работы.....	3
2. Подготовка проекта	3
3. Процесс, поток и конкурентное выполнение.....	4
4. Platform Threads.....	4
5. Почему не нужно создавать Thread на каждую задачу	5
6. ExecutorService	6
7. Future	6
8. CompletableFuture	7
9. ForkJoinPool.....	9
10. CPU-bound и blocking workload	10
11. Virtual Threads	11
12. Корректное сравнение производительности	12
13. Producer / Consumer	12
14. Завершение многопоточной программы	13
15. Практическое задание	15
15.1. Задание 1. Параллельная обработка массива	15
15.2. Задание 2. Platform Threads и Virtual Threads	15
15.3. Задание 3. Асинхронная обработка запроса.....	16
15.4. Задание 4. Producer / Consumer	16
16. Что должно быть в проекте	17
17. Требования к результатам экспериментов	17
17.1. CPU-bound эксперимент.....	17
17.2. Blocking-эксперимент.....	18
18. Вопросы для самопроверки	18
19. Будьте готовы объяснить на защите.....	18
20. Типовые ошибки	19
21. Критерии готовности работы.....	19
22. Рекомендуемые источники	19

1. Цель работы

Цель работы – освоить базовые механизмы конкурентного выполнения в Java и научиться выбирать их с учётом характера задачи. В работе рассматриваются platform threads, пулы потоков, получение отложенного результата, композиция асинхронных операций, рекурсивное распараллеливание, virtual threads и обмен данными через блокирующую очередь.

После выполнения работы студент должен:

- понимать создание и базовый жизненный цикл потока;
- использовать ExecutorService и различать Runnable и Callable<T>;
- получать, отменять и ограничивать по времени результат Future;
- строить асинхронные цепочки через CompletableFuture;
- использовать ForkJoinPool для рекурсивно делимых вычислений;
- объяснять назначение Virtual Threads и границы их применимости;
- выбирать модель выполнения для CPU-bound и blocking-задач;
- реализовывать producer/consumer через BlockingQueue;
- корректно завершать ExecutorService и обрабатывать interruption;
- сравнивать производительность нескольких реализаций в воспроизводимом учебном эксперименте.

2. Подготовка проекта

1. Проверьте установленную версию: команда `java --version` должна показывать Java 25.
2. Создайте Maven-проект и задайте для исходного кода и тестов версию языка 25.
3. Добавьте JUnit 5. Для основной части работы достаточно стандартных API Java.
4. Создайте отдельные пакеты или модули исходного кода для четырёх заданий. Не смешивайте демонстрационные классы и итоговую реализацию.
5. Запустите минимальную программу и тест через Maven, чтобы проверить конфигурацию до начала экспериментов.

Листинг 1 – Фрагмент pom.xml

```
<properties>
  <maven.compiler.release>25</maven.compiler.release>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <junit.version>5.13.4</junit.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.5.3</version>
    </plugin>
  </plugins>
</build>
```

Проверка. Выполните `mvn test`. Если Maven использует другую JDK, проверьте `JAVA_HOME` и вывод `mvn --version`. Точную patch-версию JUnit можно скорректировать по указанию преподавателя.

3. Процесс, поток и конкурентное выполнение

Процесс – выполняющийся экземпляр программы со своим адресным пространством и ресурсами операционной системы. Поток – последовательность выполнения внутри процесса. Потоки одного Java-процесса разделяют память heap, поэтому могут совместно работать с объектами, но обязаны согласовывать доступ к изменяемому состоянию.

Метод `main` выполняется в потоке `main`. Создание дополнительных потоков позволяет продвигать несколько задач независимо. Это ещё не означает, что вычисления физически идут одновременно.

Понятие	Смысл	Пример
Последовательное выполнение	Следующая задача начинается после завершения предыдущей.	А, затем В.
Конкурентность (concurrency)	Несколько задач находятся в процессе выполнения и могут чередоваться.	Один поток ожидает ввод, другой обрабатывает готовые данные.
Параллелизм (parallelism)	Несколько операций реально выполняются одновременно на разных вычислительных ресурсах.	Части массива обрабатываются на нескольких ядрах.

На рисунке 1 показано различие по временной оси. Конкурентное выполнение возможно и на одном ядре: планировщик переключается между задачами. Параллельное выполнение требует нескольких доступных вычислительных ресурсов.

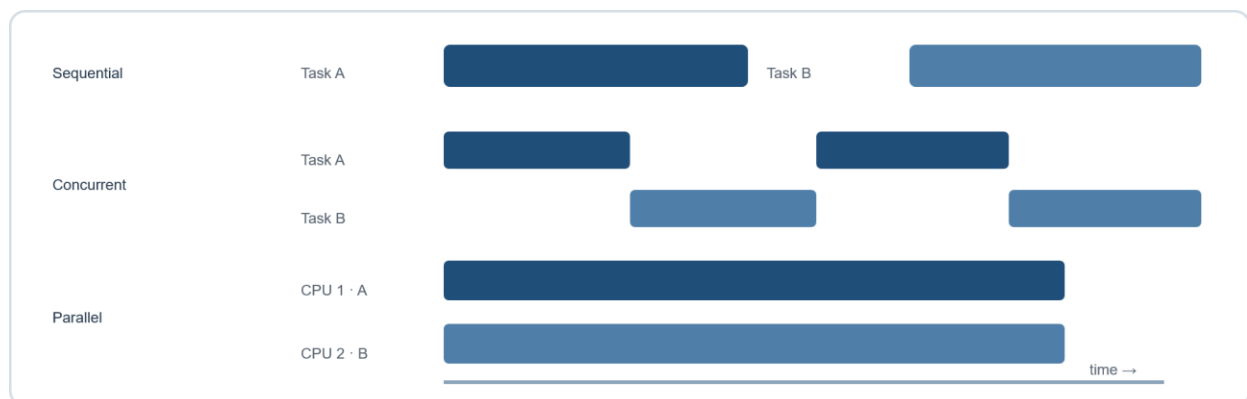


Рисунок 1 – Последовательное, конкурентное и параллельное выполнение задач

Важно. `Concurrency` $\neq$ `Parallelism`. Многопоточная программа может быть конкурентной, но в конкретный момент выполнять код только одного потока. И наоборот, параллельное вычисление – частный случай организации конкурентных задач при наличии ресурсов.

4. Platform Threads

Обычный объект `Thread` в Java представляет поток выполнения. В этом разделе под `platform thread` понимается поток Java, связанный с потоком операционной системы. Для запуска передают `Runnable` – действие без возвращаемого результата.

Листинг 2 – Создание и ожидание platform thread

```

public final class HelloThread {
    public static void main(String[] args) throws InterruptedException {
        Thread worker = new Thread(() -> {
            System.out.println(Thread.currentThread().getName());
        }, "worker-1");

        worker.start();
        worker.join();
        System.out.println("Worker finished");
    }
}

```

Вызов `start()` просит JVM начать выполнение нового потока; его метод `run()` будет вызван в этом потоке. Прямой вызов `worker.run()` – обычный вызов метода в текущем потоке, поэтому новый поток не появляется. Метод `join()` блокирует вызывающий поток до завершения `worker`.

`interrupt()` не завершает поток принудительно. Он устанавливает признак прерывания, а блокирующие методы вроде `sleep()`, `join()` или `take()` обычно реагируют исключением `InterruptedException`. Код задачи должен воспринимать прерывание как запрос на прекращение работы и выходить из контролируемого цикла либо восстанавливать `interrupt` status, если не может обработать запрос на текущем уровне.

Упрощённый жизненный цикл показан на рисунке 2. Реальное перечисление `Thread.State` подробнее, но для этой работы достаточно понимать переход от создания к выполнению, временной невозможности продолжить работу и завершению.

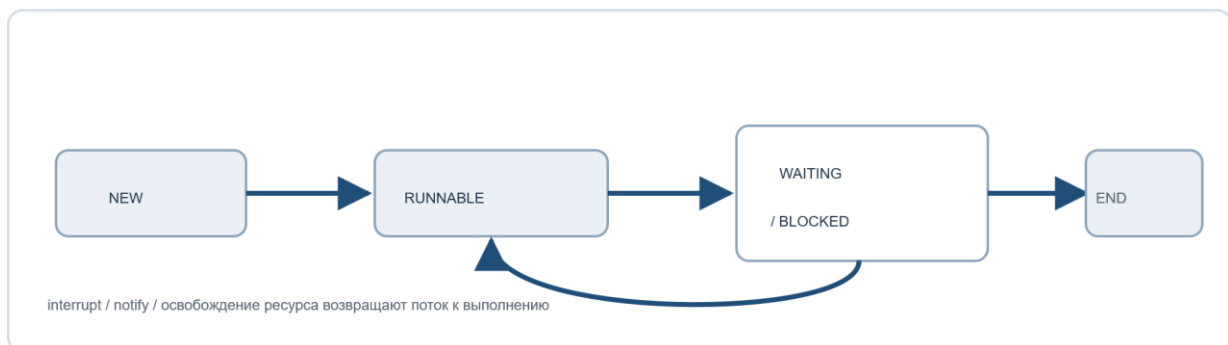


Рисунок 2 – Упрощённый жизненный цикл потока

5. Почему не нужно создавать Thread на каждую задачу

Задача и поток – разные сущности. Задача описывает работу, а поток предоставляет ресурс для её выполнения. Если для ста коротких задач создать сто новых platform threads, программа передаст операционной системе управление сотней потоков, хотя процессор сможет одновременно выполнять лишь ограниченное их число.

Создание platform thread требует памяти под стек и участия планировщика ОС. Избыточное число потоков увеличивает переключения контекста, усложняет остановку и не даёт контролировать допустимый параллелизм. Для CPU-bound нагрузки разумная отправная точка – число worker threads, сопоставимое с числом доступных процессоров. Для blocking-нагрузки допустимая конкурентность может быть выше, но её всё равно следует связывать с внешними ограничениями: соединениями, квотами, памятью и пропускной способностью сервиса.

Executor Framework разделяет постановку задач и управление потоками. Программа передаёт Runnable или Callable в исполнитель, а тот решает, когда и на каком worker thread выполнить работу.

6. ExecutorService

ExecutorService управляет жизненным циклом набора worker threads и очередью задач. Fixed thread pool ограничивает число одновременно работающих platform threads, а лишние задачи ожидают в очереди. Общая схема показана на рисунке 3.

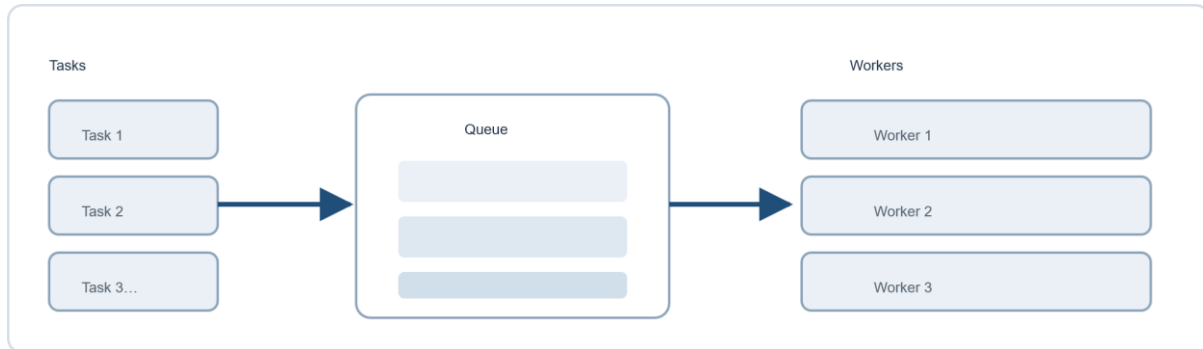


Рисунок 3 – Работа пула потоков

Листинг 3 – Отправка Callable в ExecutorService

```
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public final class ExecutorDemo {
    public static void main(String[] args) throws Exception {
        try (ExecutorService executor = Executors.newFixedThreadPool(4)) {
            Callable<Integer> task = () -> 20 + 22;
            List<Future<Integer>> futures = List.of(
                executor.submit(task), executor.submit(task));

            for (Future<Integer> future : futures) {
                System.out.println(future.get());
            }
        }
    }
}
```

Операция	Назначение
execute(Runnable)	Передаёт действие без возвращаемого значения.
submit(Runnable / Callable)	Передаёт задачу и возвращает Future, представляющий её результат или завершение.
shutdown()	Запрещает принимать новые задачи и даёт завершить уже принятые.
awaitTermination(...)	Ожидает завершения после запроса shutdown в течение заданного времени.

Runnable не возвращает значение и не объявляет checked exception в сигнатуре run(). Callable<T> возвращает T и может выбросить исключение. Исключение из Callable сохраняется в Future и проявляется при получении результата как ExecutionException.

7. Future

Future<T> – дескриптор результата задачи, которая уже отправлена на выполнение. Он позволяет узнать состояние, отменить задачу и дождаться результата. Сам Future не задаёт цепочку последующих действий.

Листинг 4 – Получение результата Future с timeout

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;

public final class FutureDemo {
    public static void main(String[] args) throws Exception {
        try (ExecutorService executor = Executors.newFixedThreadPool(2)) {
            Future<Integer> future = executor.submit(FutureDemo::calculate);
            System.out.println("Independent work");
            int result = future.get(2, TimeUnit.SECONDS);
            System.out.println(result);
        }

        private static int calculate() {
            return 6 * 7;
        }
    }
}
```

`get()` без `timeout` блокирует текущий поток до готовности результата. Ограниченное ожидание задаётся вариантом `get(long, TimeUnit)`. `isDone()` только сообщает состояние; многократная проверка `isDone()` в тесном цикле создаёт `busy waiting`. `cancel(true)` просит отменить задачу и допускает прерывание выполняющего потока, но фактическая остановка зависит от того, реагирует ли код задачи на `interruption`.

Рисунок 4 показывает типичную потерю конкурентности: после `submit()` поток `main` немедленно вызывает `get()` и ничего не делает, пока `worker` вычисляет результат. Полезнее сначала отправить независимые задачи, выполнить доступную работу и только затем ожидать те результаты, без которых продолжение невозможно.

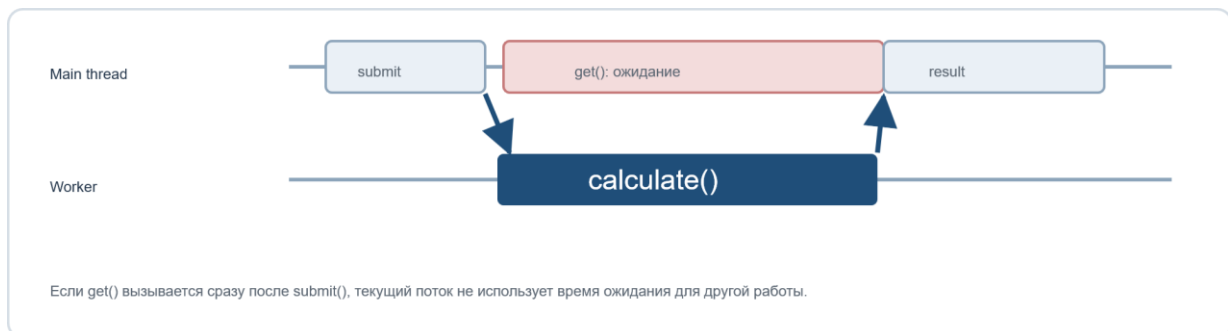


Рисунок 4 – Blocking-ожидание результата Future

8. CompletableFuture

`CompletableFuture<T>` сочетает представление будущего результата и набор операций для его преобразования. Вместо ручного ожидания между стадиями можно описать, что выполнить после успешного результата, как объединить независимые вычисления и что сделать при ошибке.

Листинг 5 – Асинхронная цепочка CompletableFuture

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public final class CompletableFutureDemo {
    public static void main(String[] args) {
        try (ExecutorService executor = Executors.newFixedThreadPool(4)) {
            CompletableFuture<String> pipeline = CompletableFuture
                .supplyAsync(CompletableFutureDemo::loadData, executor)
                .thenApply(String::trim)
                .thenCompose(data -> calculateAsync(data, executor))
                .thenApply(value -> "Result: " + value)
                .orTimeout(2, TimeUnit.SECONDS)
                .exceptionally(error -> "Error: " + error.getMessage());

            pipeline.thenAccept(System.out::println).join();
        }
    }

    private static String loadData() { return " 21 "; }

    private static CompletableFuture<Integer> calculateAsync(
        String data, ExecutorService executor) {
        return CompletableFuture.supplyAsync(
            () -> Integer.parseInt(data) * 2, executor);
    }
}
```

Метод	Форма	Когда применять
supplyAsync	$() \rightarrow T$	Асинхронная стадия возвращает значение.
runAsync	$() \rightarrow \text{void}$	Асинхронное действие без результата.
thenApply	$T \rightarrow R$	Синхронно преобразовать готовое значение.
thenCompose	$T \rightarrow \text{CompletableFuture}<R>$	Продолжить другой асинхронной операцией без вложенного Future.
thenCombine	$T + U \rightarrow R$	Объединить два независимых результата.
exceptionally	$\text{Throwable} \rightarrow T$	Сформировать резервный результат после ошибки.

thenApply соответствует обычному преобразованию $T \rightarrow R$. Если функция уже возвращает `CompletableFuture<R>`, thenApply создаст вложенную конструкцию `CompletableFuture<CompletableFuture<R>>`. thenCompose разворачивает такую зависимость в одну цепочку. Общий поток выполнения показан на рисунке 5.

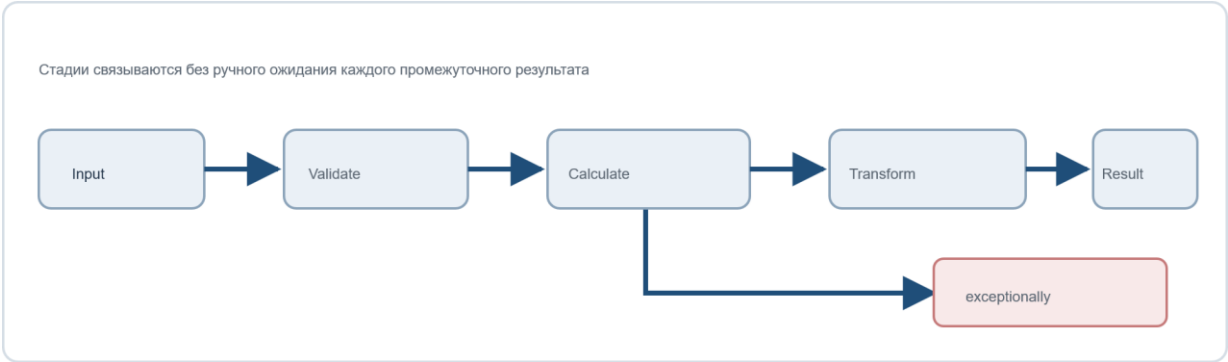


Рисунок 5 – Асинхронная цепочка CompletableFuture

Пул выполнения. Асинхронные методы без явно переданного Executor обычно используют `ForkJoinPool.commonPool()`. В учебной работе передавайте собственный Executor там, где важно контролировать ресурсы и завершение. `join()` в демонстрации нужен только для того, чтобы консольная программа не завершилась раньше цепочки; в основном задании не вызывайте `get()` или `join()` после каждого принятого запроса.

9. ForkJoinPool

Fork/Join предназначен для CPU-bound задач, которые можно рекурсивно разделить на независимые части, обработать и объединить. `RecursiveTask<V>` возвращает значение, `RecursiveAction` – нет. При достижении порога задача выполняется последовательно, иначе создаются подзадачи.

Листинг 6 – Компактный пример `RecursiveTask`

```
import java.util.concurrent.ForkJoinPool;
import java.util.concurrent.RecursiveTask;

public final class ForkJoinSumDemo {
    private static final int THRESHOLD = 1_000;

    static final class SumTask extends RecursiveTask<Long> {
        private final int[] values;
        private final int from;
        private final int to;

        SumTask(int[] values, int from, int to) {
            this.values = values;
            this.from = from;
            this.to = to;
        }

        @Override protected Long compute() {
            if (to - from <= THRESHOLD) {
                long sum = 0;
                for (int i = from; i < to; i++) sum += values[i];
                return sum;
            }
            int middle = (from + to) >> 1;
            SumTask left = new SumTask(values, from, middle);
            SumTask right = new SumTask(values, middle, to);
            left.fork();
            long rightResult = right.compute();
            return left.join() + rightResult;
        }
    }

    public static void main(String[] args) {
        int[] values = {1, 2, 3, 4, 5, 6};
        long sum = ForkJoinPool.commonPool()
            .invoke(new SumTask(values, 0, values.length));
        System.out.println(sum);
    }
}
```

`fork()` планирует подзадачу для конкурентного выполнения, `join()` получает её результат. В примере одна половина отправляется в пул, а другая вычисляется текущим worker thread. Это уменьшает лишние ожидания. Декомпозиция и обратная сборка показаны на рисунке 6.

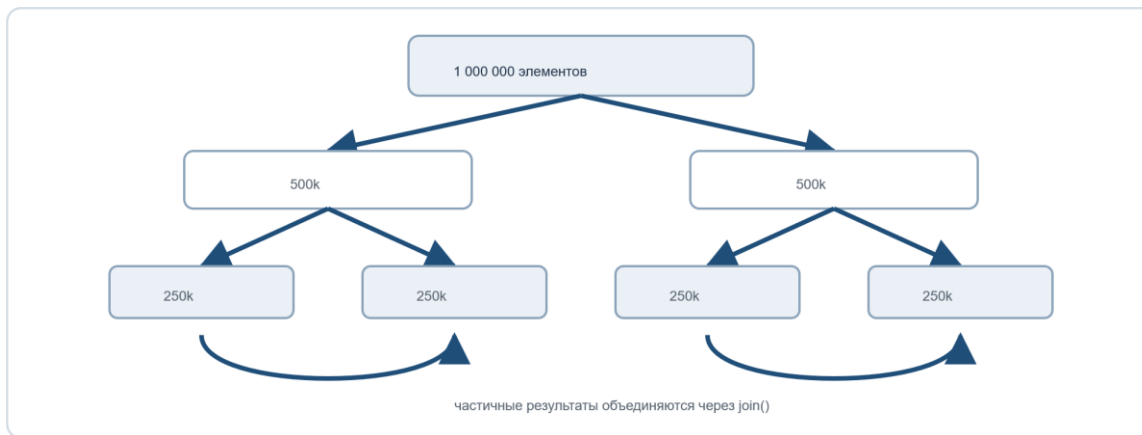


Рисунок 6 – Разделение и сборка задачи в Fork/Join

ForkJoinPool применяет work stealing: свободный worker может забрать подзадачу из очереди другого worker. Порог разбиения влияет на результат: слишком крупный оставляет мало параллельной работы, слишком маленький увеличивает накладные расходы. В задании порог нужно подобрать экспериментально, не копируя число из демонстрации.

10. CPU-bound и blocking workload

Выбор механизма начинается с ответа на вопрос: где задача проводит основное время? CPU-bound задача почти постоянно использует процессор. Blocking-задача значительную часть времени ждёт внешний ресурс и в это время не выполняет полезные инструкции на CPU.

Характеристика	CPU-bound	Blocking / I/O-like
Типичные примеры	Математические вычисления, обработка массива, хеширование, компрессия.	HTTP, база данных, файл, сокет, ожидание внешнего ответа.
Основное ограничение	Число ядер и производительность CPU.	Задержка внешнего ресурса, лимиты соединений, пропускная способность.
Разумная конкурентность	Обычно сопоставима с числом доступных процессоров.	Может быть заметно выше числа ядер, если задачи большую часть времени ждут.
Подходящий механизм	Ограниченный пул, ForkJoinPool для делимых вычислений.	Virtual threads или контролируемый пул platform threads.
Опасная ошибка	Создать сотни вычисляющих потоков и ожидать линейного ускорения.	Игнорировать лимиты внешней системы и запустить неограниченное число операций.

На рисунке 7 тёмный участок означает работу CPU, светлый – ожидание. У blocking-задачи есть возможность освободить carrier/platform thread на период ожидания; у непрерывного вычисления такой паузы нет.

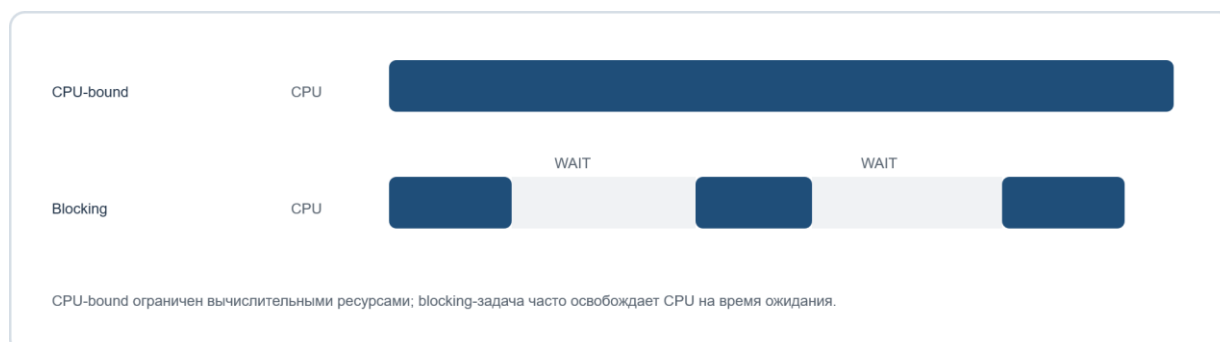


Рисунок 7 – CPU-bound и blocking-задачи

Практический вывод. Для CPU-bound эксперимента нельзя добавлять Thread.sleep(): он превращает измерение вычисления в модель ожидания. Искусственная задержка допустима во втором задании именно потому, что там моделируется blocking I/O.

11. Virtual Threads

Virtual thread – объект Thread с точки зрения Java API, но JVM может обслуживать множество виртуальных потоков небольшим числом carrier/platform threads. Виртуальный поток дешёв относительно platform thread и удобен для модели «одна задача – один поток», когда задач много и они часто блокируются на поддерживаемых операциях ожидания.

Листинг 7 – Запуск задач в virtual threads

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;

public final class VirtualThreadDemo {
    public static void main(String[] args) throws Exception {
        Thread.startVirtualThread(() -> System.out.println("virtual"))
            .join();

        try (ExecutorService executor =
            Executors.newVirtualThreadPerTaskExecutor()) {
            List<Future<Integer>> futures = new ArrayList<>();
            for (int id = 0; id < 100; id++) {
                int taskId = id;
                futures.add(executor.submit(() -> blockingTask(taskId)));
            }
            for (Future<Integer> future : futures) {
                System.out.println(future.get());
            }
        }

        private static int blockingTask(int id) throws InterruptedException {
            Thread.sleep(50); // имитация ожидания внешнего ресурса
            return id;
        }
    }
}
```

newVirtualThreadPerTaskExecutor() создаёт новый virtual thread для каждой отправленной задачи и не является пулом с фиксированным числом worker threads. Это не означает отсутствие ограничений: приложение по-прежнему должно контролировать число соединений, объём памяти и нагрузку на внешние системы.

Virtual threads повышают пропускную способность систем с большим количеством ожидающих задач, но не уменьшают время отдельной операции и не добавляют вычислительных ядер. Если тысяча virtual threads одновременно выполняет CPU-bound расчёт, реально считать будут лишь несколько – примерно по числу доступных процессоров. Концептуальное отличие показано на рисунке 8.

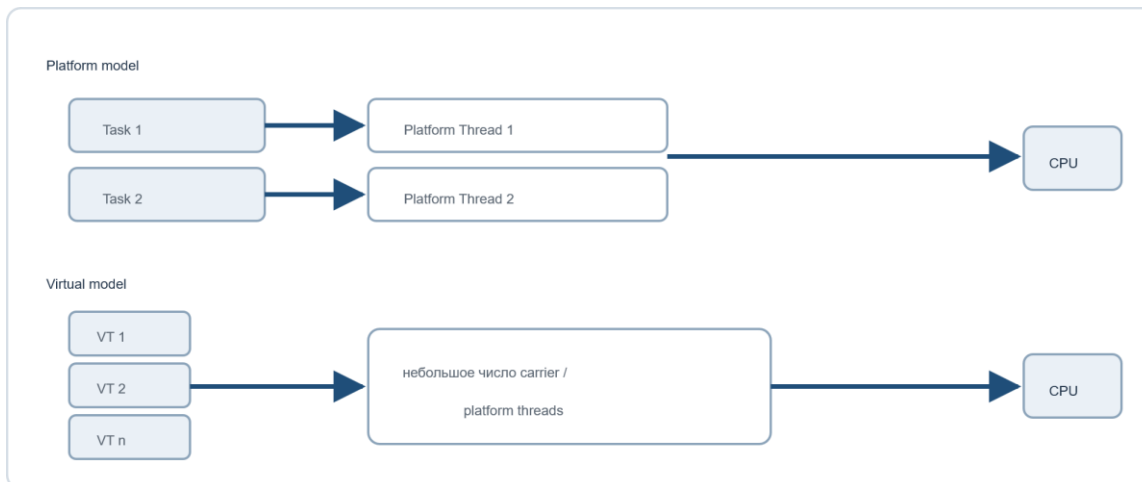


Рисунок 8 – Platform Threads и Virtual Threads

12. Корректное сравнение производительности

Один запуск не даёт надёжного результата: JVM прогревает код, JIT-компилятор оптимизирует часто выполняемые участки, на измерение влияют фоновые процессы и сборка мусора. `System.currentTimeMillis()` имеет недостаточную точность для коротких интервалов; для измерения длительности используйте разность `System.nanoTime()`.

Для учебного сравнения соблюдайте одинаковые условия:

- одни и те же входные данные для всех реализаций;
- несколько warm-up запусков, результаты которых не входят в таблицу;
- несколько измеряемых запусков в одной JVM или по одинаковому сценарию;
- один компьютер, одна версия JDK и одинаковые параметры запуска;
- проверка совпадения вычисленного результата до сравнения времени;
- среднее или медиана измеряемых запусков с указанием выбранной метрики.

Реализация	Запуск 1	Запуск 2	Запуск 3	Запуск 4	Запуск 5	Среднее / медиана
Последовательная						
ExecutorService						
ForkJoinPool						

Граница метода. Это учебный сравнительный эксперимент, а не полноценный microbenchmark. Для профессионального микробенчмаркинга JVM применяется JMH; в данной работе JMH использовать не требуется.

13. Producer / Consumer

В схеме producer/consumer одни компоненты создают задачи, а другие обрабатывают их с собственной скоростью. Очередь отделяет момент создания от момента обработки и задаёт безопасный канал передачи между потоками.

`BlockingQueue` является потокобезопасной очередью. `put()` ожидает свободное место в ограниченной очереди, а `take()` ожидает появление элемента. За счёт этого не нужен цикл с `queue.isEmpty()`, который постоянно расходует CPU и всё равно подвержен гонке между проверкой и извлечением.

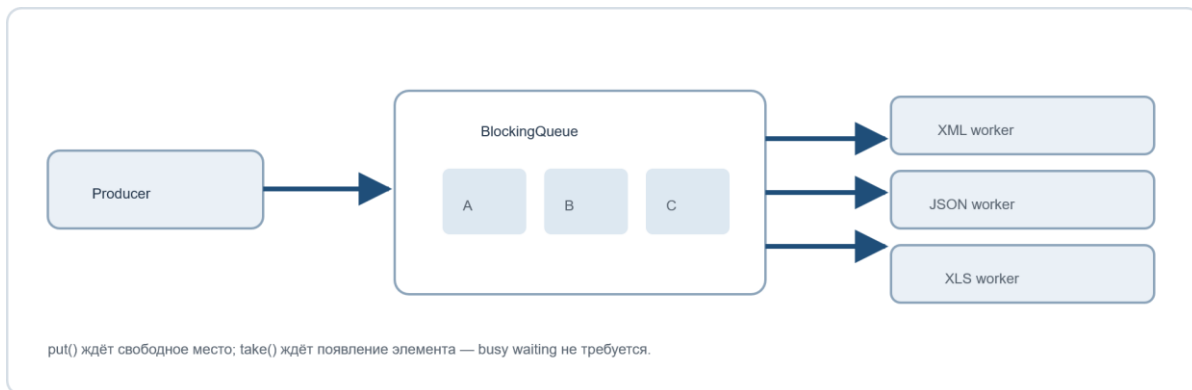


Рисунок 9 – Producer/Consumer с BlockingQueue

Листинг 8 – Ограниченная BlockingQueue и сигнал завершения

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public final class BlockingQueueDemo {
    private static final String STOP = "<stop>";

    public static void main(String[] args) throws InterruptedException {
        BlockingQueue<String> queue = new ArrayBlockingQueue<>(3);
        Thread consumer = Thread.startVirtualThread(() -> {
            try {
                String item;
                while (!(item = queue.take()).equals(STOP)) {
                    System.out.println("Processed: " + item);
                }
            } catch (InterruptedException error) {
                Thread.currentThread().interrupt();
            }
        });

        queue.put("A");
        queue.put("B");
        queue.put(STOP);
        consumer.join();
    }
}
```

Для нескольких consumers нужен согласованный протокол завершения: например, по одному специальному маркеру на каждый consumer либо отдельная координация через executor. Маркер не должен совпадать с корректной задачей предметной области. В основном задании выберите конечное число файлов и добейтесь, чтобы каждый ID встретился в обработанной статистике ровно один раз.

14. Завершение многопоточной программы

Корректная программа прекращает принимать новые задачи, завершает уже принятые, освобождает ресурсы и не оставляет незавершённых non-daemon потоков. shutdown() инициирует мягкое завершение ExecutorService. shutdownNow() пытается прервать выполняющиеся задачи и возвращает список ещё не начатых, но не гарантирует мгновенную остановку.

Листинг 9 – Graceful shutdown для ExecutorService

```

import java.time.Duration;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.TimeUnit;

public final class ExecutorsSupport {
    private ExecutorsSupport() { }

    public static void shutdownGracefully(
        ExecutorService executor, Duration timeout) {
        long millis = timeout.toMillis();
        executor.shutdown();
        try {
            if (!executor.awaitTermination(millis, TimeUnit.MILLISECONDS)) {
                executor.shutdownNow();
                if (!executor.awaitTermination(millis, TimeUnit.MILLISECONDS)) {
                    System.err.println("Executor did not terminate");
                }
            }
        } catch (InterruptedException error) {
            executor.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}

```

При перехвате `InterruptedException` нельзя оставлять пустой `catch`. Если текущий уровень не завершает операцию полностью, восстановите признак прерывания через `Thread.currentThread().interrupt()` и выйдите из работы. Последовательность мягкого завершения показана на рисунке 10.

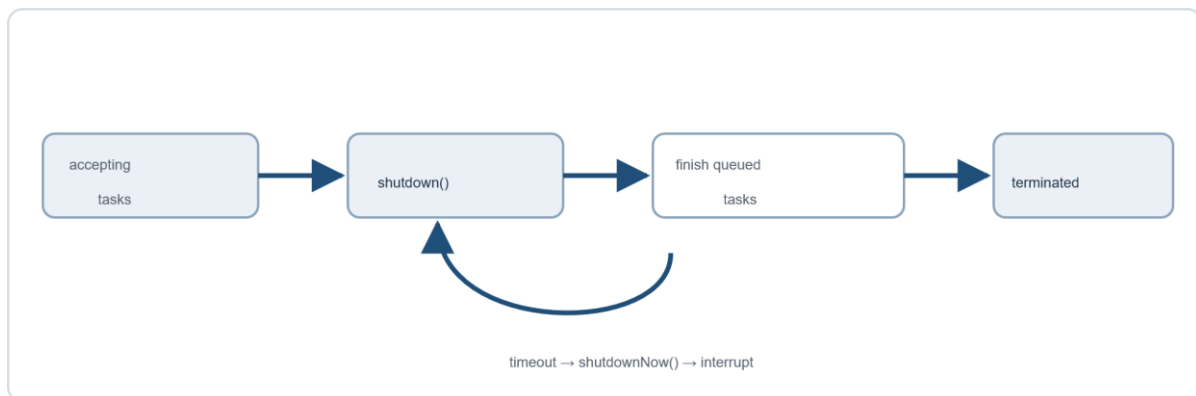


Рисунок 10 – Корректное завершение *ExecutorService*

Не используйте. Бесконечный `while (true)`, который прекращается только закрытием терминала, не считается корректным завершением. У программы должен быть достижимый конечный сценарий и понятный протокол остановки `producer`, `consumers` и `executors`.

15. Практическое задание

Основное задание состоит из четырёх связанных частей. Для каждой части подготовьте отдельную точку запуска либо набор тестов. Демонстрационные листинги показывают API, но не задают архитектуру итогового решения.

15.1. Задание 1. Параллельная обработка массива

Создайте большой массив целых чисел. Размер подберите так, чтобы вычисление занимало измеримое время на вашем компьютере, но не приводило к нехватке памяти. Массив должен быть создан один раз и использоваться всеми сравниваемыми реализациями.

Выполняемая операция определяется по номеру студента в списке:

вариант = ((номер студента по списку - 1) % 3) + 1

Вариант	Операция
1	Сумма элементов
2	Максимальный элемент
3	Минимальный элемент

Реализуйте три варианта алгоритма:

1. последовательная обработка;
2. обработка через ExecutorService;
3. обработка через ForkJoinPool.

- Разделяйте массив на непересекающиеся диапазоны. Каждый элемент должен участвовать в вычислении ровно один раз.
- Для ExecutorService самостоятельно выберите число worker threads и объясните выбор.
- Для ForkJoinPool задайте порог последовательной обработки и проверьте несколько значений.
- Перед измерением убедитесь, что три реализации возвращают одинаковый результат.
- Выполните warm-up и не менее пяти измеряемых запусков каждого варианта.
- Внутри вычисления запрещено использовать Thread.sleep(). Это CPU-bound эксперимент.

В выводе укажите, какая реализация оказалась быстрее, во сколько раз, почему результат мог отличаться от ожиданий и как он связан с Runtime.getRuntime().availableProcessors(). Не обещайте линейное ускорение: разбиение и объединение результатов также имеют стоимость.

15.2. Задание 2. Platform Threads и Virtual Threads

Смоделируйте 500–1000 независимых blocking-задач. Каждая задача выполняет небольшой объём обычной работы, затем ожидает внешний ресурс в течение случайного интервала в заданном диапазоне и возвращает результат. Заранее сформируйте одинаковый набор параметров задач, чтобы оба варианта получили одни и те же задержки.

Сравните:

1. Executors.newFixedThreadPool(...) с обоснованным размером пула;
 2. Executors.newVirtualThreadPerTaskExecutor().
- Измерьте общее время обработки полного набора задач.
 - Зафиксируйте число задач, размер fixed pool и диапазон задержек.

- Проведите несколько запусков в одинаковых условиях.
- Проверьте, что все задачи вернули результат и программа завершилась.

Допустимая имитация. Искусственная задержка здесь допустима, потому что используется для моделирования blocking I/O. Не переносите этот приём в задание 1.

Объяснение должно связывать результат с тем, что platform thread занят на время blocking-ожидания, тогда как virtual thread может быть временно размонтирован с carrier thread. Формулировка «Virtual Threads быстрее, потому что они современные» не объясняет наблюдение.

15.3. Задание 3. Асинхронная обработка запроса

Реализуйте обработку входных запросов через `CompletableFuture`. Запрос содержит ID и данные, достаточные для выбранной вами простой предметной операции. Цепочка должна соответствовать следующей логике:

Input → Validation → Calculation → Transformation → Result

- несколько запросов могут обрабатываться одновременно;
- основной поток не ожидает каждый запрос перед приёмом следующего;
- ошибка валидации или вычисления преобразуется в понятный результат либо регистрируется;
- у цепочки присутствует `timeout`;
- результат выводится после завершения соответствующей цепочки и содержит ID запроса;
- используемый `Executor` корректно завершается после всех цепочек.

Самостоятельно решите, какие стадии являются обычными преобразованиями `thenApply`, а какие возвращают новую асинхронную операцию и требуют `thenCompose`. Для двух независимых источников данных можно применить `thenCombine`. Не превращайте решение в последовательный цикл с `get()` после каждого `submit()`.

15.4. Задание 4. Producer / Consumer

Реализуйте конечную модель обработки файловых задач. `FileTask` содержит уникальный ID, тип XML / JSON / XLS и размер от 10 до 100 условных единиц. `Producer` создаёт заранее заданное число задач с различными интервалами и помещает их в ограниченную `BlockingQueue`.

`Consumers` получают задачи из очереди и выполняют обработку, длительность которой зависит от размера. Допустимы два варианта архитектуры: отдельный обработчик для каждого типа либо общий пул обработчиков с маршрутизацией. Выбранный вариант опишите в README или комментариях к схеме классов.

- используется потокобезопасная ограниченная очередь;
- отсутствует `busy waiting`;
- `interruption` обрабатывается явно;
- каждый `FileTask` обрабатывается один раз;
- `producer` и `consumers` имеют конечный протокол завершения;
- программа не требует ручного завершения через `Ctrl+C`;
- счётчики статистики корректны при конкурентном обновлении.

После завершения выведите:

Листинг 10 – Формат итоговой статистики producer/consumer

```
Generated: ...
Processed: ...
XML: ...
JSON: ...
XLS: ...
Total processing time: ...
```

Проверьте инварианты: $\text{Generated} = \text{Processed}$; $\text{Processed} = \text{XML} + \text{JSON} + \text{XLS}$; множество обработанных ID совпадает с множеством сгенерированных ID. Если producer работает быстрее consumers, ограниченная очередь должна естественно создавать backpressure за счёт блокировки put().

16. Что должно быть в проекте

Рекомендуемая структура отделяет четыре задания и тесты. Имена пакетов не обязательны, если выбранная вами архитектура столь же понятна.

Листинг 11 – Рекомендуемая структура исходного кода

```
src
├── main/java
│   ├── task1
│   ├── task2
│   ├── task3
│   └── task4
└── test/java
    ├── task1
    ├── task2
    ├── task3
    └── task4
```

В репозитории должны находиться pom.xml, исходный код, тесты и краткий README с командами запуска. Не добавляйте каталоги target, настройки IDE и крупные сгенерированные файлы. Для воспроизводимости зафиксируйте seed генератора случайных данных либо сохраните входной набор.

17. Требования к результатам экспериментов

Для заданий 1 и 2 представьте таблицы измерений и короткое собственное объяснение. Укажите единицы времени, количество запусков, характеристики нагрузки и число доступных процессоров. Не удаляйте неудачные результаты без объяснения: разброс сам по себе является наблюдением.

17.1. CPU-bound эксперимент

Реализация	Среднее / медиана	Относительное ускорение
Последовательная		1,00
ExecutorService		
ForkJoinPool		

Относительное ускорение можно вычислить как время последовательной реализации, делённое на время сравниваемой. Значение меньше 1 означает замедление. Такое возможно на малом массиве или при слишком мелком разбиении.

17.2. Blocking-эксперимент

Executor	Количество задач	Параметры	Общее время
Fixed thread pool		Размер пула: ...	
Virtual thread per task		Диапазон ожидания: ...	

Поясните, где именно находится blocking operation и почему при такой нагрузке большое число ожидающих virtual threads не равно большому числу одновременно выполняющихся вычислений.

18. Вопросы для самопроверки

1. Чем процесс отличается от потока?
2. Чем конкурентность отличается от параллелизма?
3. Чем вызов `run()` отличается от `start()`?
4. Для чего используется `join()`?
5. Почему создание отдельного platform thread для каждой задачи может быть плохой идеей?
6. Для чего используется `ExecutorService`?
7. Чем `Runnable` отличается от `Callable<T>`?
8. Что происходит при вызове `Future.get()`?
9. Чем `Future` отличается от `CompletableFuture`?
10. Для чего используется `ForkJoinPool`?
11. Что такое work stealing?
12. Чем CPU-bound задача отличается от blocking-задачи?
13. Почему увеличение числа потоков не ускоряет CPU-bound задачу бесконечно?
14. Для чего нужны Virtual Threads?
15. Ускорят ли Virtual Threads вычисление большой матрицы на одном и том же CPU? Почему?
16. Для чего используется `BlockingQueue`?
17. Что произойдет при `put()` в заполненную ограниченную `BlockingQueue`?
18. Почему многопоточная программа должна корректно обрабатывать interruption и завершение?

19. Будьте готовы объяснить на защите

- Почему для задания 1 не использовалась искусственная задержка?
- Почему `ForkJoin` подходит для вашей операции и как выбран порог разбиения?
- Что изменится, если создать thread pool размером 1000 для CPU-bound задачи?
- Почему Virtual Threads хорошо показали себя именно во втором эксперименте?
- Где в вашей программе находится blocking operation?
- Что произойдет при немедленном вызове `get()` после каждого `submit()`?
- Чем `thenApply` отличается от `thenCompose` в вашей цепочке?
- Что произойдет, если producer работает быстрее consumer?
- Почему `BlockingQueue` лучше ручной проверки `queue.isEmpty()` в цикле?
- Как программа гарантирует, что задача обрабатывается один раз?
- Как и в каком порядке завершаются ваши `ExecutorService`, producer и consumers?

20. Типовые ошибки

Ошибка	Почему это проблема
Вызов <code>run()</code> вместо <code>start()</code>	Код выполняется в текущем потоке; конкурентного запуска нет.
Огромное число <code>platform threads</code>	Растут расходы памяти и планирования, контроль ресурсов теряется.
<code>Future.get()</code> сразу после каждого <code>submit()</code>	Текущий поток последовательно ждёт каждую задачу.
Нет <code>shutdown()</code>	Executor может удерживать JVM и ресурсы после завершения полезной работы.
Пустой <code>catch</code> для <code>InterruptedException</code>	Запрос на остановку теряется, завершение становится непредсказуемым.
<code>Thread.sleep()</code> внутри CPU benchmark	Измеряется ожидание, а не вычислительная нагрузка.
Измерение одного запуска	Результат зависит от прогрева JVM и случайного фонового влияния.
Разные входные данные	Сравниваются неэквивалентные вычисления.
Обычная <code>Queue</code> или <code>ArrayList</code> между потоками	Возникают гонки данных и нарушение внутреннего состояния коллекции.
Busy waiting	Поток расходует CPU на постоянную проверку состояния.
Бесконечный <code>while (true)</code>	Нет управляемого протокола завершения.
Virtual Threads ускоряют любые задачи	CPU-bound вычисление всё равно ограничено числом процессоров.

21. Критерии готовности работы

Перед защитой проверьте:

- все четыре задания запускаются из чистой копии репозитория;
- результаты последовательной и параллельных реализаций задания 1 совпадают;
- CPU-bound эксперимент проведён несколько раз на одинаковых данных;
- blocking-эксперимент сравнивает `fixed pool` и `virtual threads` на одном наборе задач;
- `CompletableFuture` не превращён в последовательный код постоянными `get()` или `join()`;
- `producer/consumer` не использует `busy waiting` и не теряет задачи;
- `interruption` не игнорируется;
- все executors и потоки корректно завершаются;
- таблицы измерений заполнены, единицы времени указаны;
- вы можете объяснить результаты через характер нагрузки, число ядер и накладные расходы.

22. Рекомендуемые источники

Для уточнения поведения API используйте официальную документацию Java 25:

1. Thread – Java SE 25 API	6. ForkJoinPool – Java SE 25 API
2. ExecutorService – Java SE 25 API	7. BlockingQueue – Java SE 25 API
3. Executors – Java SE 25 API	8. Java 25 Core Libraries Developer Guide
4. Future – Java SE 25 API	9. OpenJDK JMH
5. CompletableFuture – Java SE 25 API	